CS 313E · ELEMENTS OF SOFTWARE DESIGN

Asymptotic Notation

Dr. Veena Ravishankar

Laptops open.

ACTIVITY · YOU DO THIS

Warm-up: from last class

Last class we worked out, for insertion sort:

best case T(n) = a n + b

worst case T(n) = a n^2 + b n + c

(a, b, c are constants built from the line costs C1 .. C9)

Which statement about the WORST case is true?

A T(n) = O(n^3)

B T(n) = O(n^2)

C T(n) = O(n) because a, b, c are constants

D T(n) = O(n^2) only if a = 1

Step 1
Contemplate and then vote.

Step 2
Then we discuss it together.

By the end of today you can…

AND HERE IS HOW IT CONNECTS TO LAST CLASS

1. **Tell O, Omega and Theta apart** — Upper bound, lower bound, and tight bound - and which one actually answers "how fast is it".

2. **Produce the constants for a bound** — Give c and n0 - or c1, c2 and n0 - and check the inequality holds.

3. **Read the Big O straight off a loop** — One loop, nested loops, and halving - and recognise which one you are looking at.

Last class Big O let us say insertion sort is no worse than quadratic. Today we add the other two bounds, so we can say how fast it is, not just how slow it is not.

Growth of functions, and asymptotic analysis

The three notations, and what each one claims.

Rate of growth

--The running time we are interested in, in an algorithm analysis, is named the rate of growth or the order of growth.

Asymptotic analysis

Mathematical notation used to describe bounds on a function's growth as the input size approaches infinity: O gives an upper bound, Omega a lower bound, and Theta a tight bound.

Why it is the right question

An algorithm that is asymptotically more efficient is the best choice for all but very small inputs.

The three notations

Oh (O), Omega (Ω) and Theta (Θ).

Big O

An upper bound. f grows no faster than g.

Omega

A lower bound. f grows at least as fast as g.

Theta

A tight bound - above and below at once. f grows at the same rate as g.

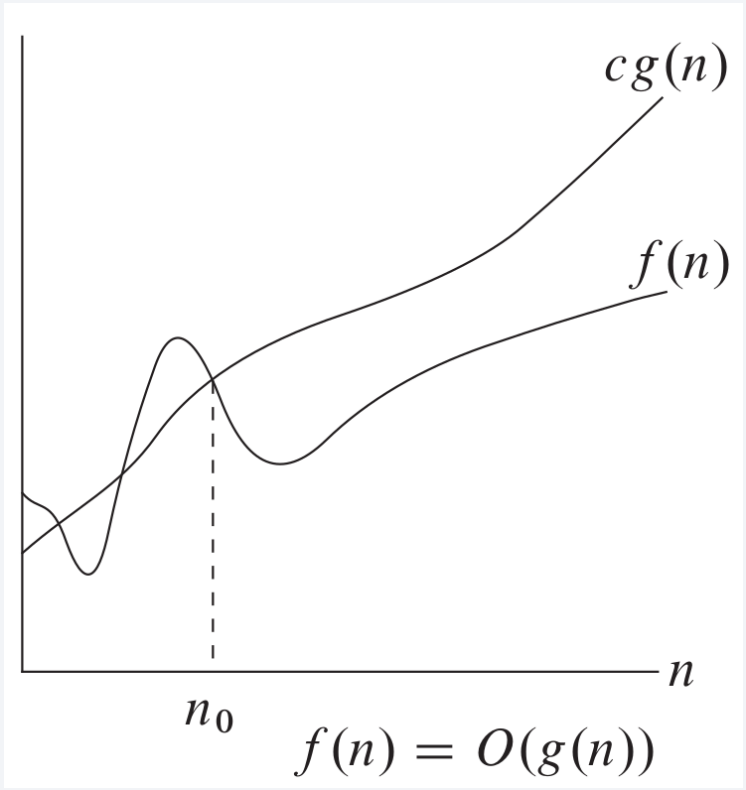
And two strict versions

little o is a strictly smaller upper bound, little omega a strictly larger lower bound.

The one to aim for

Theta is the preferred notation as it gives more precise information because it supplies both an upper and a lower bound.

Big O: asymptotic upper bound



DEFINITION

Big O

O(g(n)) = { f(n) : there exist constants c, n0 > 0 such that 0 ≤ f(n) ≤ c g(n) for all n ≥ n0 }.

In words

f(n) = O(g(n)) means f grows no faster than g. It characterizes an upper bound on the asymptotic behaviour of a function, within a constant factor.

What the picture says

At and to the right of n_0, the value of f(n) always lies on or below c x g(n). What happens before n0 does not matter.

Examples of Big O

f(n) = 4n^2 - 8n + 4

```
f(n) = O(n^2)
    because for c = 4,
    f(n) <= 4n^2 when n > 1


f(n) = O(n^3)
    because for c = 1,
    f(n) < n^3 when n > 1
```

BUT NOT O(n)

```
f(n) is NOT O(n)

because for any c > 0,
f(n)< cn?
```

One function, many upper bounds

O(n^2) and O(n^3) are both true. Big O is an upper bound, not an exact answer - which is why the tight bound gets its own notation.

Big O is an upper bound, so a function has many of them. Saying 4n^2 - 8n + 4 is O(n^3) is true, just not useful.

ACTIVITY · YOU DO THIS

Predict, then run: which bounds hold?

Work it out on paper.

Given

$f(n) = 4n^2 + 100n + 500$

$g(n) = n^2$

and separately

$h(n) = n^3 - 100\,n^2$

Which of the two is $O(n^2)$?

Step 1 (2-3 mins)
Discuss with neighbor and then vote.

Which statement is true?

A both are $O(n^2)$

B neither is $O(n^2)$

C f is $O(n^2)$; h is not

D h is $O(n^2)$; f is not

Order of growth of functions

1 < log n < sqrt(n) < n < n log n < n^2 < n^3 < ... < 2^n < 3^n < ... < n^n

Constant
O(c) - the work does not depend on n at all.

Logarithmic
O(log n) - each step throws away a fixed fraction of what is left. BST search on a balanced tree.

Linear
O(n) - one pass over the input. Insertion sort on an already sorted array.

Log linear
O(n log n) - the best any comparison sort can do. Merge sort, next class.

Quadratic
O(n^2) - a loop inside a loop. Insertion sort on a reverse sorted array.

Cubic
O(n^3) - three nested loops. Naive matrix multiplication.

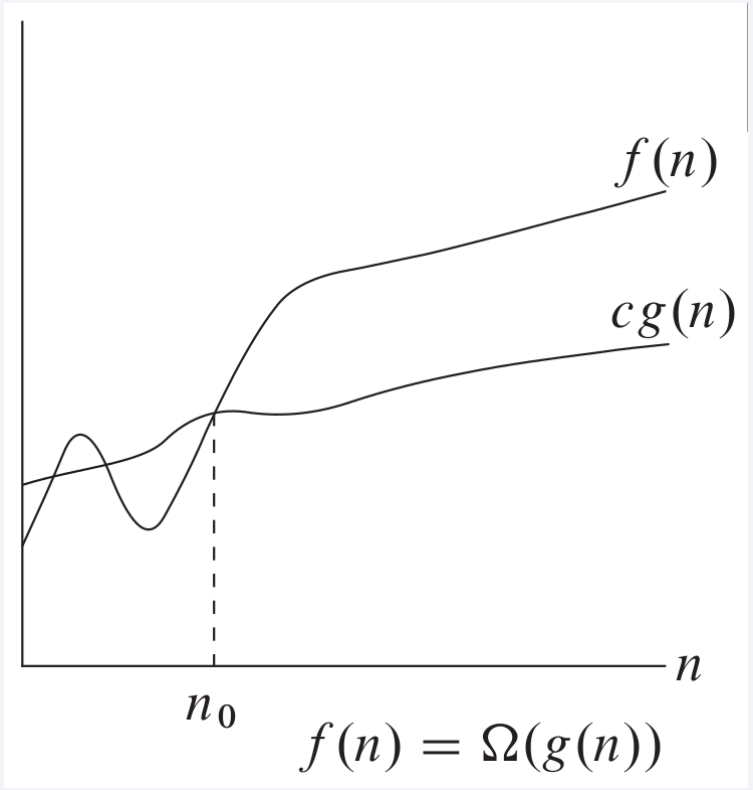
Exponential
O(2^n) - adding one element doubles the work. Trying every subset.

The two rules of thumb
Every computer scientist knows two rules of thumb about asymptotics: logarithms grow more slowly than polynomials, and polynomials grow more slowly than exponentials.

Function Name	Big O
Constant	$O(c)$
Linear	$O(n)$
Logarithmic	$O(log(n))$
Log Linear	$O(nlog(n))$
Quadratic	$O(n^2)$
Cubic	$O(n^3)$
Exponential	$O(2^n)$

Why the ranking is the point
Once two algorithms sit in different rows of this list, no amount of constant-factor tuning moves one above the other for large n.

Omega: asymptotic lower bound



DEFINITION

Omega

Omega(g(n)) = { f(n) : there exist constants c, n0 > 0 such that 0 ≤ c g(n) ≤ f(n) for all n ≥ n0 }.

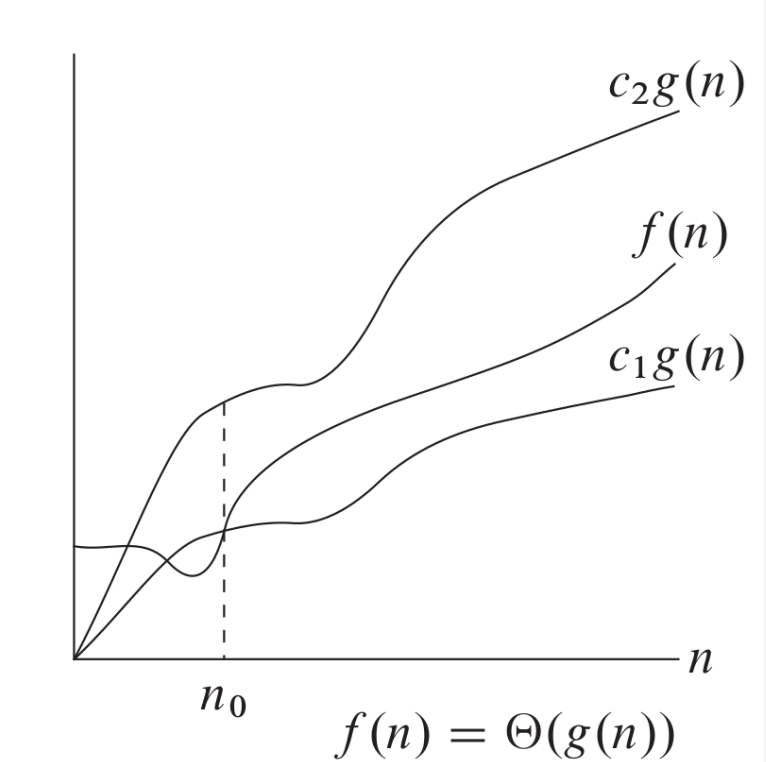
In words

Big O provides an asymptotic upper bound. An asymptotic LOWER bound is provided by Omega. We write f(n) = Omega(g(n)) if at and to the right of n0, the value of f(n) always lies on or above c x g(n).

Same picture, flipped

Compare with the Big O figure: the curve c x g(n) has moved from above f to below it. Everything else about the definition is identical.

Theta: asymptotic tight bound



DEFINITION

Theta

Theta(g(n)) = { f(n) : there exist constants c1, c2, n0 > 0 such that 0 ≤ c1 g(n) ≤ f(n) ≤ c2 g(n) for all n ≥ n0 }.

In words

Theta asymptotically bounds a function from above and below, to within constant factors. At and to the right of n0, f(n) always lies between c1 g(n) and c2 g(n) inclusive.

Two constants, not one

That is the only structural difference from O and Omega - you have to produce c1 AND c2. f = Theta(g) exactly when f = O(g) and f = Omega(g) both hold.

Things to remember

A BOUND IS A CLAIM. YOU PROVE IT BY PRODUCING THE CONSTANTS.

`O is "no faster than"`	So a function has infinitely many Big O bounds. n^2 is O(n^2), O(n^3), O(2^n) - all true, only the first useful.
`Theta is O and Omega together`	If you can give c1, c2 and n0, you have pinned the growth rate exactly. That is the statement worth making.
`To DISPROVE a bound, contradict it`	Assume the constants exist, divide through by g(n), and show the result cannot hold for arbitrarily large n. That is how 6n^3 is shown not to be Theta(n^2).

Every proof on the next slides is the same two moves: write the inequality, then divide both sides by g(n) and read off what the constants have to be.

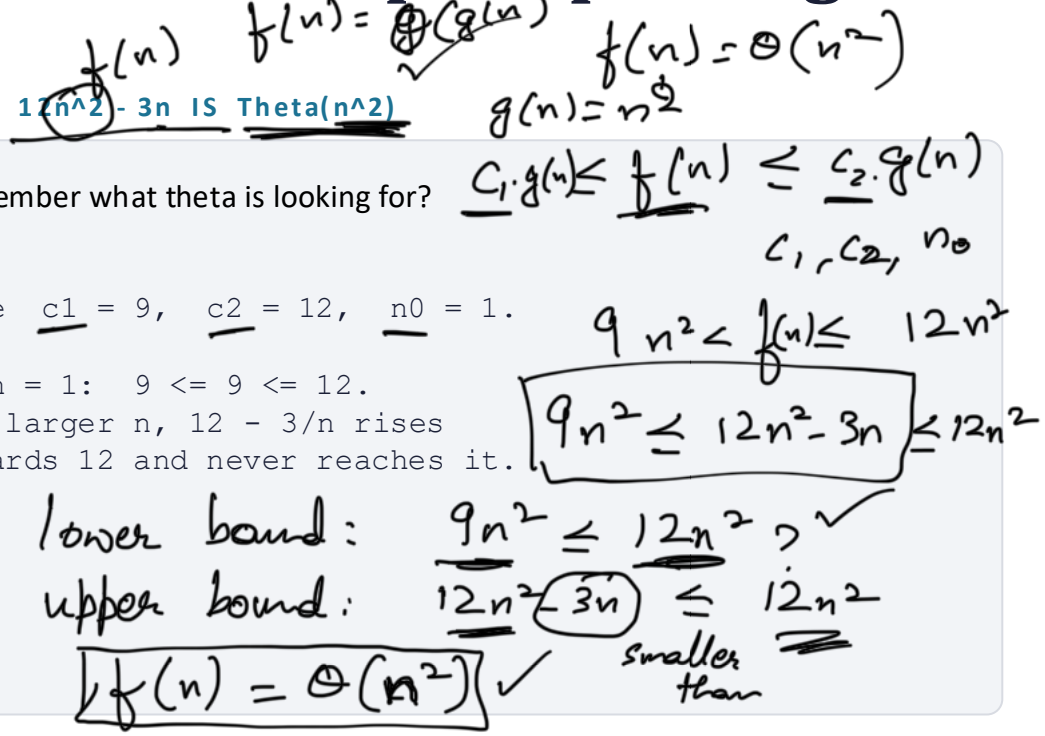
Worked examples: proving and disproving Theta

CLAIM: 12n^2 - 3n IS Theta(n^2)

Remember what theta is looking for?

Take c1 = 9, c2 = 12, n0 = 1.

At n = 1: 9 <= 9 <= 12.
For larger n, 12 - 3/n rises towards 12 and never reaches it.



Other constants work too

c1 = 9 and c2 = 12 is one choice, not the ONLY choice. Any c1 ≤ 9 and c2 ≥ 12 does the job, and a larger n0 lets you tighten c1.

CLAIM: 6n^3 IS NOT Theta(n^2)



Proof by contradiction.

Suppose c2 and n0 exist with

6n^3 <= c2 n^2 for all n >= n0

divide both sides by n^2

n <= c2 / 6

which is not true for arbitrarily large n, since c2 is a constant.

The disproof is the same move

Write the inequality, divide by g(n), and look at what is left. If a constant has to be bigger than something that grows, there is no such constant.

Second example from CLRS, page 46.

Examples: O, Omega and Theta

Each one is the same recipe - name g(n), write the inequality, produce the constants.

Is f(n) = 2n^2 + 4n = Omega(n) ?

Yes. Take g(n) = n. Show 0 ≤ c g(n) ≤ f(n) for n > n0. With n0 = 1 and c = 2: 0 ≤ 2 x 1 ≤ 2 + 4. Holds.

Is f(n) = 2n^2 + 4n = Theta(n^2) ?

Yes. Take g(n) = n^2. Show 0 ≤ c1 g(n) ≤ f(n) ≤ c2 g(n) for n > n0. With n0 = 1, c1 = 2, c2 = 8: 0 ≤ 2 ≤ 6 ≤ 8. Holds.

Is f(n) = 2^(3+n) = O(2^n) ?

Yes. Take g(n) = 2^n. Then 2^(3+n) = 2^3 x 2^n = 8 x 2^n, so with n0 = 1 and c = 8 the inequality 2^(3+n) ≤ c x 2^n holds with equality.

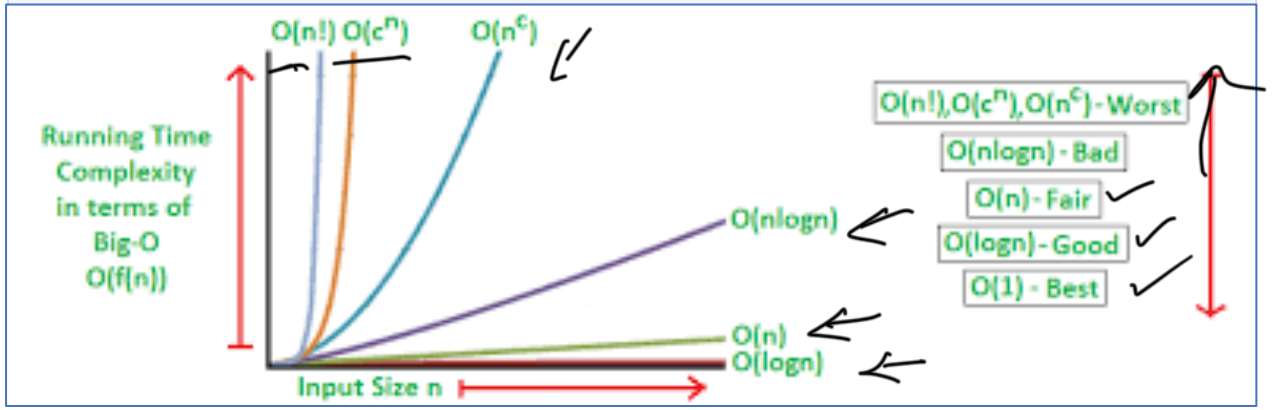
Any quadratic is Theta(n^2) ?

The "any c" is the whole difference

For O and Omega you get to CHOOSE one convenient c.

Common functions

Function Name	Big O
Constant	$O(c)$
Linear	$O(n)$
Logarithmic	$O(log(n))$
Log Linear	$O(nlog(n))$
Quadratic	$O(n^2)$
Cubic	$O(n^3)$
Exponential	$O(2^n)$



WHAT TO TAKE FROM THE TABLE

The ranking is the point

Once two algorithms sit in different rows, no amount of constant-factor tuning moves one above the other for large n.

Where each one comes from

One pass is linear. A loop inside a loop is quadratic. Halving the problem each step is logarithmic. Trying every subset is exponential.

The two rules of thumb

Logarithms grow more slowly than polynomials, and polynomials grow more slowly than exponentials.

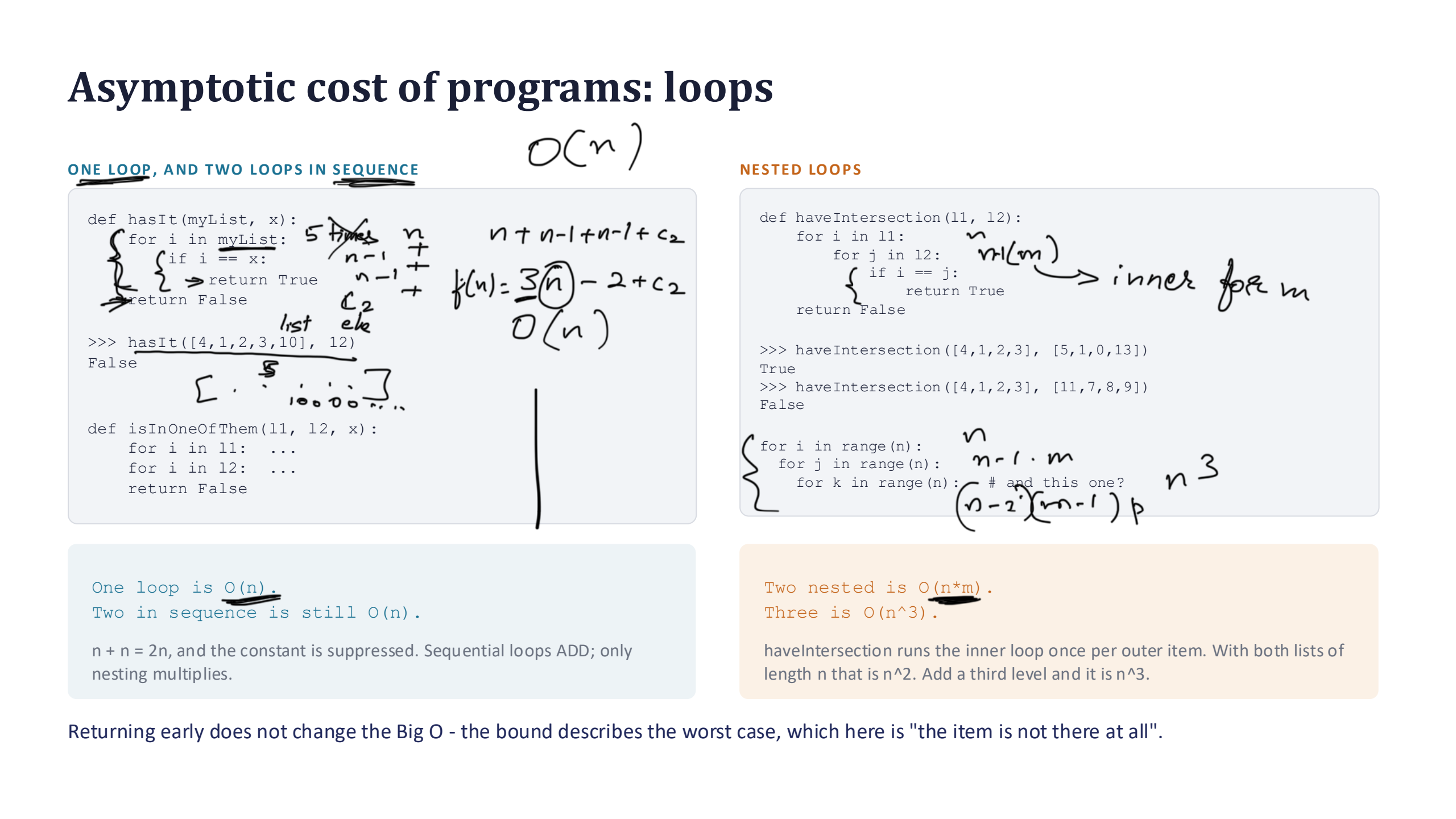
Asymptotic cost of programs: loops

ONE LOOP, AND TWO LOOPS IN SEQUENCE

```
def hasIt(myList, x):
    for i in myList:
        if i == x:
            return True
    return False

>>> hasIt([4,1,2,3,10], 12)
False

def isInOneOfThem(l1, l2, x):
    for i in l1:  ...
    for i in l2:  ...
    return False
```



One loop is O(n).
Two in sequence is still O(n).

n + n = 2n, and the constant is suppressed. Sequential loops ADD; only nesting multiplies.

NESTED LOOPS

```
def haveIntersection(l1, l2):
    for i in l1:
        for j in l2:
            if i == j:
                return True
    return False

>>> haveIntersection([4,1,2,3], [5,1,0,13])
True
>>> haveIntersection([4,1,2,3], [11,7,8,9])
False

for i in range(n):
    for j in range(n):
        for k in range(n):  # and this one?
```

Two nested is O(n*m).
Three is O(n^3).

haveIntersection runs the inner loop once per outer item. With both lists of length n that is n^2. Add a third level and it is n^3.

Returning early does not change the Big O - the bound describes the worst case, which here is "the item is not there at all".

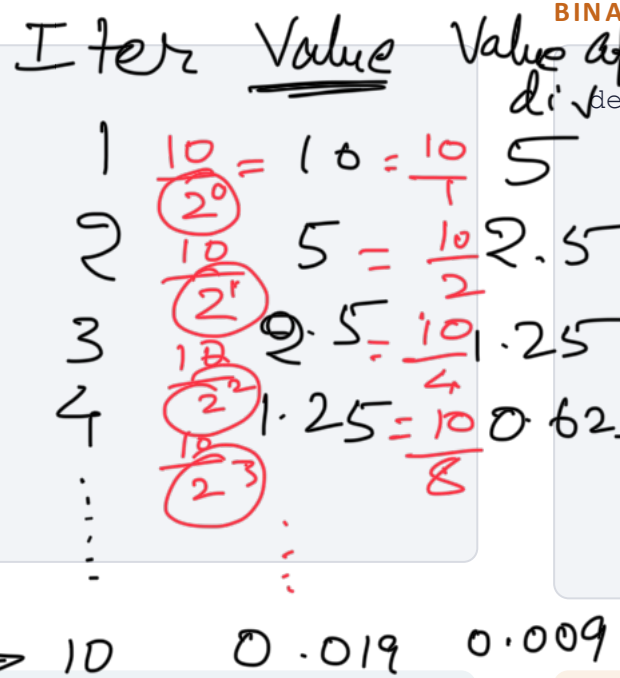
O(log n): halving, and binary search

HOW MANY TIMES CAN YOU DIVIDE BY 2?

```
def doIt(x):
    while x > 0.01:
        print(x)
        x = x / 2
    return x

>>> doIt(10)
10   5.0   2.5   1.25   0.625
0.3125   0.15625   0.078125
0.0390625   0.01953125
```

$\log_2(n) = k$ means $2^k = n$



$n = 2^k$ $\log n = k$

$x = \frac{n}{2^k}$ After k iter, current value (x) = Starting (n) / 2^k

Ten lines for 10

Each step halves x, so the number of steps is how many times n can be halved before it gets small - that is log2(n).

BINARY SEARCH

```
def bSearch(data, value):
    left = 0
    right = len(data) - 1

    while left <= right:
        middle = (left + right) // 2
        if value < data[middle]:
            right = middle - 1
        elif value > data[middle]:
            left = middle + 1
        else:
            return middle
    print("Not in the list")
```

Three probes for nine items

bSearch on [1..9] looking for 6 prints counts 0, 1, 2 and returns index 5. Doubling the list to 18 items adds one probe, not nine.

Same shape again: my_log$_2$N(100) prints 1 2 4 8 16 32 64 and returns 7 - because 2^7 is the first power of two at or above 100.

ACTIVITY · YOU DO THIS

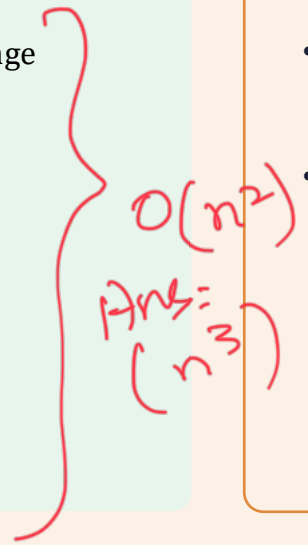
Programming activity: what is the Big O?

Work with the person next to you and run everything you write.

CORE

Programming problem. Everyone attempts it.

```
n = 5  # example value, you can change
for i in range(0, n):
    count = 0
    x = i * n
    print("i, x:", i, x)
    for j in range(0, x):
        count += 1
    print("count:", count)
```



STRETCH

If you finish early.

- Write my_log2N(n): count how many times i doubles before reaching n. Check that my_log2N(100) returns 7
- Write bSearch(data, value) and print the probe count. How many probes for a list of 1000 random numbers?

TAKE-HOME

- Add a third nested loop to haveIntersection. Measure the running time at n = 100, 200, 400 and see whether it behaves like n^3
- Prove, with constants, that 4n^2 + 100n + 500 = O(n^2)

Step 1 (5-6 mins)
Write. Partner allowed. Run it.

Step 2
We look at the solution together.

Today, in four lines

- O is an upper bound, Omega a lower bound, Theta both at once - and Theta is the real answer.
- To prove a bound, produce the constants. To disprove one, divide by g(n) and contradict.
- little o and little omega are the strict versions: the inequality must hold for EVERY c.
- Sequential loops add, nested loops multiply, and halving gives log n.

BEFORE NEXT CLASS

- find_Intersection - finish it
- my_log2N and bSearch with probe counts
- Prove 4n^2 + 100n + 500 = O(n^2)
- CLRS Sec 3.1 - asymptotic notation
- CLRS Sec 3.1 and 3.2 - standard notations
- algorist.com/algowiki - more Big O proofs

Announcement

- Exam 1, 7p, OCT 28, JGB 2.324
- Topics: everything after Exam1 until analysis of algorithms (included)—until the end of today
- On paper, 75 mins
- Note sheet: 1 handwritten or printed A4 size
- Closed book, closed notes
- D&A letter—RegisterBlast
- PSS2 tonight at 5p, GDC 2.216

ACTIVITY · YOU DO THIS

Additional: the growing inner loop (additional)

The inner loop does not run n times. It runs a different number of times every pass.

```
for i in range(0, n):
    count = 0
    x = i * n
    print("i, x:", i, x)

    for j in range(0, x):
        count += 1

    print("count", count)


What is the Big O of this code?

Hint: check how many times the
inner loop runs.
```

```
Add up the inner loop runs:

f(n) = 0 + n + 2n + 3n + ...
       + n(n-1)

     = n (0 + 1 + 2 + ... + (n-1))

and the sum in brackets is

     (n-1)n / 2 ,  which is
     Theta(n^2)

so

     f(n) = Theta(n^3)

     and f(n) = O(n^3).
```

Reference: definitions from today

Reference for studying. Full wording, kept from the original slides.

Asymptotic analysis
An analysis of how an algorithm performs as its input grows larger, without bounds. An algorithm that is asymptotically more efficient is the best choice for all but very small inputs.

Big O - asymptotic upper bound
O(g(n)) = { f(n) : there exist constants c, n0 > 0 such that 0 ≤ f(n) ≤ c g(n) for all n ≥ n0 }. f grows no faster than g.

Omega - asymptotic lower bound
Omega(g(n)) = { f(n) : there exist constants c, n0 > 0 such that 0 ≤ c g(n) ≤ f(n) for all n ≥ n0 }. At and to the right of n0, f lies on or above c x g(n).

Theta - asymptotic tight bound
Theta(g(n)) = { f(n) : there exist constants c1, c2, n0 > 0 such that 0 ≤ c1 g(n) ≤ f(n) ≤ c2 g(n) for all n ≥ n0 }. Bounds a function from above and below, to within constant factors.

little o
o(g(n)) = { f(n) : for ANY positive constant c > 0 there exists n0 > 0 such that 0 ≤ f(n) < c g(n) for all n ≥ n0 }. A stricter, less-than upper bound. 2n = o(n^2), but 2n^2 is not.

little omega
omega(g(n)) = { f(n) : for ANY positive constant c > 0 there exists n0 > 0 such that 0 ≤ c g(n) < f(n) for all n ≥ n0 }. A stricter, greater-than lower bound. n^2/2 = omega(n), but not omega(n^2).

Transitivity
If f = Theta(g) and g = Theta(h) then f = Theta(h) - and the same for O, Omega, little o and little omega.

Reflexivity and symmetry
f = Theta(f), f = O(f), f = Omega(f). Symmetry: f = Theta(g) if and only if g = Theta(f).

Transpose symmetry
f = O(g) if and only if g = Omega(f). f = o(g) if and only if g = omega(f).

Thank you!

Questions on Ed. Office hours on Canvas.